Khushi Shukla-D15A/62

Experiment No 10: Autoencoder for Image Denoising using CNN (MNIST Dataset)

Aim

To design and implement a Convolutional Autoencoder using deep learning techniques to remove noise from images and reconstruct clean images using the MNIST dataset.

Theory

An **Autoencoder** is an unsupervised neural network used for learning efficient representations of data, typically for dimensionality reduction and reconstruction.

It consists of two main components:

1. Encoder

The encoder compresses the input image into a lower-dimensional latent representation:

$$z = f(x) = \sigma(Wx + b)$$

2. Decoder

The decoder reconstructs the original image from the compressed representation:

$$\hat{x} = g(z) = \sigma(W'z + b')$$

Denoising Autoencoder

In this experiment, a **denoising autoencoder** is used.

Instead of feeding clean images, we:

- Add noise to images
- Train the model to recover original images

Noisy Image → Encoder → Latent Space → Decoder → Clean Image

Why Convolutional Autoencoder?

Unlike simple dense networks, **Convolutional Neural Networks (CNNs)**:

- Capture spatial features
- Preserve image structure
- Provide better reconstruction quality

1. Dataset Source

- **Dataset Name**: MNIST Dataset
- **Source**: Kaggle
- **Link**: https://www.kaggle.com/datasets/hojjatk/mnist-dataset

2. Dataset Description

- Total images: **70,000**
- Training images: **60,000**
- Testing images: **10,000**
- Image size: **28 × 28 pixels**
- Color format: **Grayscale**

Characteristics:

- Images of handwritten digits (0–9)
- Simple and clean dataset
- Ideal for image processing tasks

3. Mathematical Formulation

Convolution Operation:
$Z = X * W + b$

Where:

- (X) = input image
- (W) = filter/kernel
- (b) = bias

Activation Function (ReLU):

$f(x) = \max(0, x)$

Loss Function (Binary Crossentropy):

$$L = -\sum [x \log(\hat{x}) + (1 - x) \log(1 - \hat{x})]$$

Objective:

Minimize reconstruction error:

$$\min ||x - \hat{x}||$$

4. Algorithm Limitations

- Cannot perfectly recover heavily corrupted images
- May produce slightly blurred outputs
- Requires proper tuning of layers and filters
- Computationally expensive for large datasets

5. Methodology / Workflow

Steps:

1. Load MNIST dataset
2. Normalize pixel values
3. Add noise to images
4. Build convolutional autoencoder
5. Train model on noisy images
6. Reconstruct clean images
7. Evaluate performance

Workflow Diagram:

Original Image → Add Noise → Noisy Image
↓
Encoder (CNN)
↓
Latent Space
↓
Decoder (CNN)
↓
Reconstructed Image

6. Code Implementation

```
import numpy as np, matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input,Conv2D,MaxPooling2D,UpSampling2D
from tensorflow.keras.callbacks import EarlyStopping

(x_train,_),(x_test,_)=mnist.load_data()
x_train=x_train.astype('float32')/255.;x_test=x_test.astype('float32')/255.
x_train=x_train.reshape(len(x_train),28,28,1);x_test=x_test.reshape(len(x_test),28,28,1)

noise_factor=0.3
x_train_noisy=np.clip(x_train+noise_factor*np.random.normal(size=x_train.shape),0.,1.)
x_test_noisy=np.clip(x_test+noise_factor*np.random.normal(size=x_test.shape),0.,1.)

input_img=Input(shape=(28,28,1))
x=Conv2D(32,(3,3),activation='relu',padding='same')(input_img)
x=MaxPooling2D((2,2),padding='same')(x)
x=Conv2D(16,(3,3),activation='relu',padding='same')(x)
encoded=MaxPooling2D((2,2),padding='same')(x)
x=Conv2D(16,(3,3),activation='relu',padding='same')(encoded)
x=UpSampling2D((2,2))(x)
x=Conv2D(32,(3,3),activation='relu',padding='same')(x)
x=UpSampling2D((2,2))(x)
decoded=Conv2D(1,(3,3),activation='sigmoid',padding='same')(x)

autoencoder=Model(input_img,decoded)
autoencoder.compile(optimizer='adam',loss='binary_crossentropy')

early_stop=EarlyStopping(patience=3,restore_best_weights=True)

history=autoencoder.fit(x_train_noisy,x_train,epochs=20,batch_size=128,validation_data=(x_test
_noisy,x_test),callbacks=[early_stop])
```

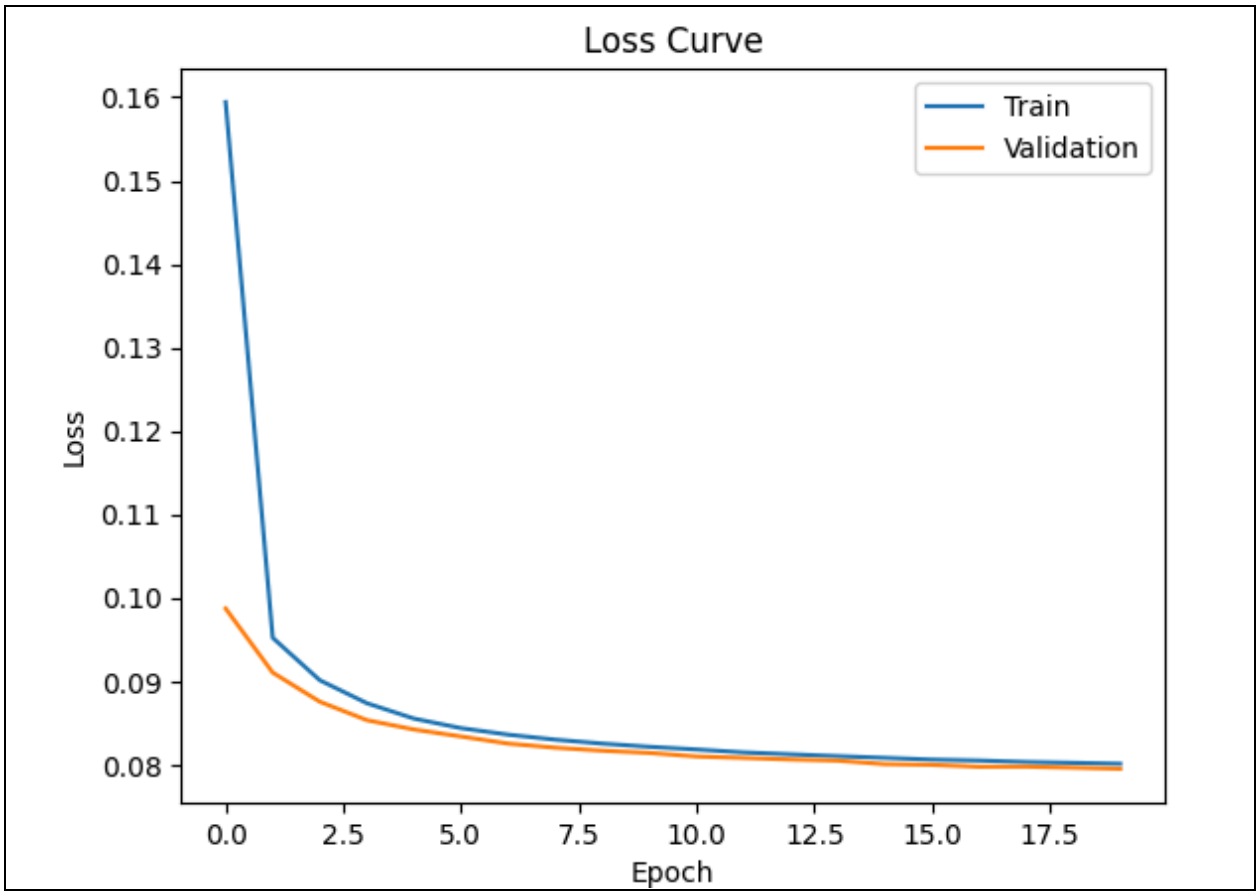
```
decoded_imgs=autoencoder.predict(x_test_noisy)

# 🔥 LOSS GRAPH
plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Loss Curve")
plt.xlabel("Epoch");plt.ylabel("Loss")
plt.legend(["Train","Validation"])
plt.show()

# 🔥 RANDOM VISUALIZATION
idx=np.random.randint(0,len(x_test),5)
plt.figure(figsize=(12,6))
for i in range(5):

plt.subplot(3,5,i+1);plt.imshow(x_test_noisy[idx[i]].reshape(28,28),cmap='gray');plt.title("Noisy
");plt.axis("off")

plt.subplot(3,5,i+6);plt.imshow(x_test[idx[i]].reshape(28,28),cmap='gray');plt.title("Original");pl
t.axis("off")

plt.subplot(3,5,i+11);plt.imshow(decoded_imgs[idx[i]].reshape(28,28),cmap='gray');plt.title("De
noised");plt.axis("off")
plt.show()
# 🔥 DIFFERENCE IMAGE (NOISE REMOVED)
plt.figure(figsize=(10,3))
for i in range(3):
    diff=np.abs(x_test[idx[i]]-decoded_imgs[idx[i]])
    plt.subplot(1,3,i+1)
    plt.imshow(diff.reshape(28,28),cmap='hot')
    plt.title("Difference")
    plt.axis("off")
plt.show()
```

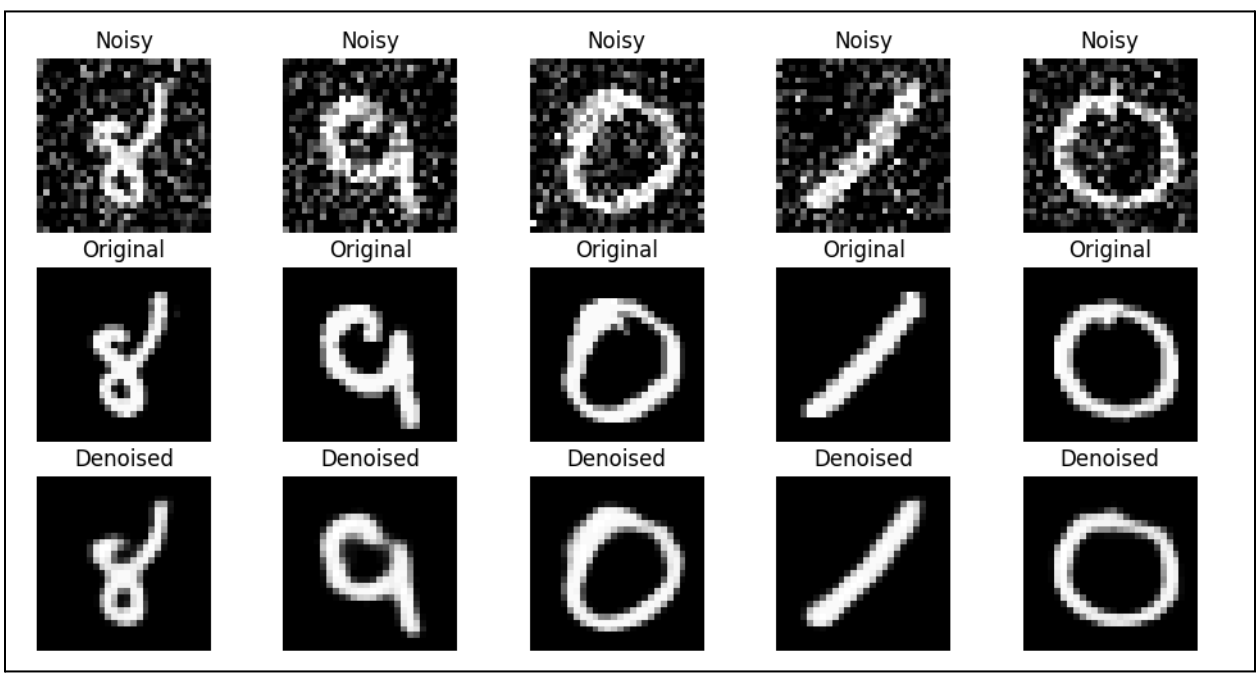
7. Output

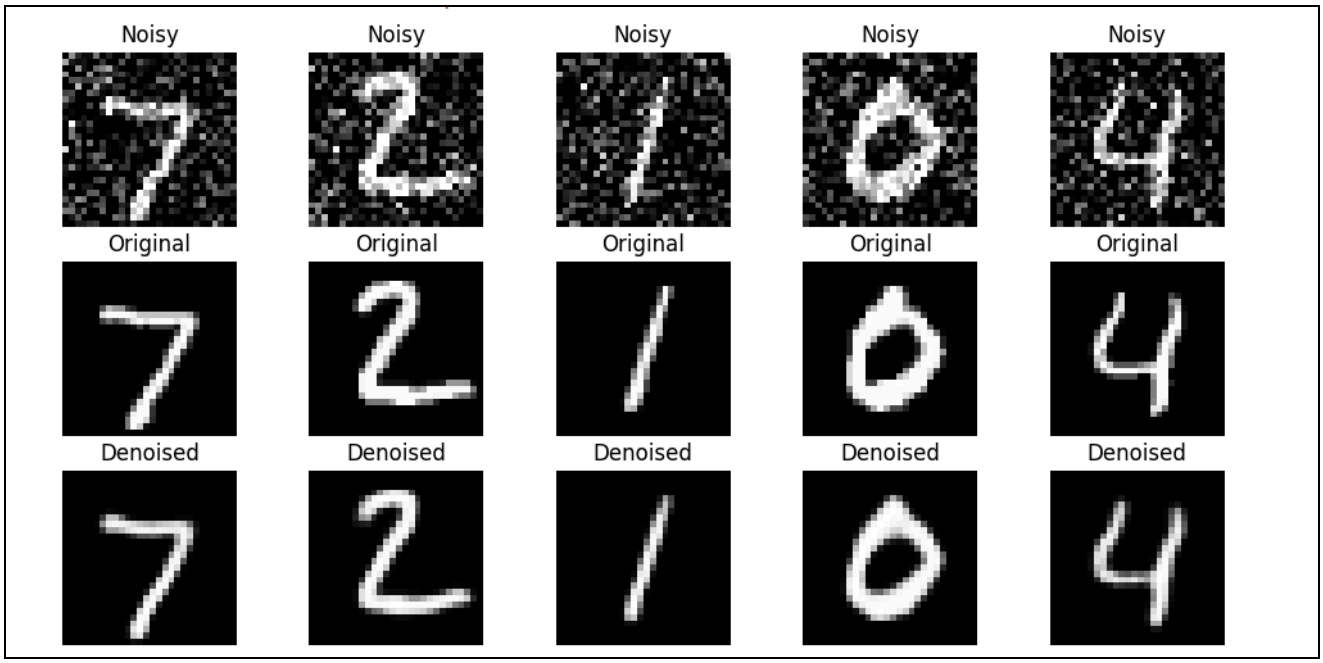
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 96s 201ms/step - loss: 0.1594 - val_loss: 0.0987
Epoch 2/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 95s 202ms/step - loss: 0.0952 - val_loss: 0.0910
Epoch 3/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 93s 199ms/step - loss: 0.0901 - val_loss: 0.0876
Epoch 4/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 92s 196ms/step - loss: 0.0874 - val_loss: 0.0853
Epoch 5/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 92s 196ms/step - loss: 0.0855 - val_loss: 0.0842
Epoch 6/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 143s 200ms/step - loss: 0.0844 - val_loss: 0.0834
Epoch 7/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 92s 195ms/step - loss: 0.0836 - val_loss: 0.0825
Epoch 8/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 92s 197ms/step - loss: 0.0830 - val_loss: 0.0820
Epoch 9/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 144s 200ms/step - loss: 0.0825 - val_loss: 0.0817
Epoch 10/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 94s 201ms/step - loss: 0.0822 - val_loss: 0.0814
Epoch 11/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 93s 198ms/step - loss: 0.0818 - val_loss: 0.0810
Epoch 12/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 142s 198ms/step - loss: 0.0815 - val_loss: 0.0808
Epoch 13/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 93s 198ms/step - loss: 0.0813 - val_loss: 0.0806
Epoch 14/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 93s 197ms/step - loss: 0.0810 - val_loss: 0.0805
Epoch 15/20
```

Epoch 15/20

```
469/469 ━━━━━━━━━━━━━━━━━━━━ 92s 197ms/step - loss: 0.0808 - val_loss: 0.0801
Epoch 16/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 93s 199ms/step - loss: 0.0806 - val_loss: 0.0800
Epoch 17/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 94s 201ms/step - loss: 0.0805 - val_loss: 0.0798
Epoch 18/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 94s 201ms/step - loss: 0.0803 - val_loss: 0.0798
Epoch 19/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 140s 196ms/step - loss: 0.0802 - val_loss: 0.0797
Epoch 20/20
469/469 ━━━━━━━━━━━━━━━━━━━━ 92s 196ms/step - loss: 0.0801 - val_loss: 0.0795
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 12ms/step
```


Loss Curve
Train
Validation
Loss
Epoch
0.16
0.15
0.14
0.13
0.12
0.11
0.10
0.09
0.08
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5


Noisy
Noisy
Noisy
Noisy
Noisy
Original
Original
Original
Original
Original
Denoised
Denoised
Denoised
Denoised
Denoised



8. Performance Analysis

- Loss decreases steadily during training
- Model learns to extract important features
- Denoised images preserve digit structure

Key Insights:

- CNN-based autoencoder performs better than dense autoencoder
- Some minor blurring remains due to reconstruction

9. Hyperparameter Tuning

Parameter	**Value**
Epochs	20
Batch Size	128
Noise Factor	0.3
Filters	32, 16
Kernel Size	3×3

Impact:

- Lower noise → better output
- More epochs → improved learning
- More filters → better feature extraction

Conclusion

In this experiment, a convolutional autoencoder was successfully implemented to remove noise from images using the MNIST dataset. The model effectively reconstructed clean images from noisy inputs, demonstrating the power of deep learning in image processing tasks. The use of convolutional layers significantly improved performance compared to traditional dense autoencoders. This approach has practical applications in fields such as medical imaging, computer vision, and signal processing.